

Código Fuente

OVERPASS TURBO	3
Comercios generales	3
Gastronomía	3
Mercados y supermercados	4
Hoteles y hospedajes	4
Industria	5
Salud y Transporte	5
POSTGIS	7
Extensión postgis	7
Tabla de capas point por radio censal	7
Tabla de capas polygon por radio censal	11
Tabla de información estadística por radio censal	12
Tabla integral de capas point, poligonales y datos estadísticos por radio censal	14
Test variable nueva: tabla de capas puntuales + radio de cercanía 500 m por radio censal	15
Test variable nueva: tabla de capas puntuales + búfer de cercanía 200 m por radio censal	21
Test variable nueva: tabla de cantidad de metros de avenida por radio censal	28
Test variable nueva: tabla de cantidad de bancos sobre avenidas por radio censal + búfer de 500m	29
Test variable nueva: tabla de cantidad de bancos BNA por radio censal	31
Tabla integral dataset final	32
Exportación de tabla resumen	34
Importación de tabla predicciones	35
Modificación de tabla predicciones a datos geográficos	35
R	37
Preparación de ambiente	37
Carga de paquetes y librerías	37
Carga de dataset local	38
Exploración y modificación dataset	38
Gráficos boxplot completos por fuente de información	40
Estadísticas generales	41
Gráficos boxplot ordenados por intervalos	42
Análisis de correlaciones	44
Preparación de dataset para modelos	46
Análisis predictivo por regresión lineal simple - MODELO 1	47
Análisis predictivo por regresión lineal simple con PCE - MODELO 2	48

Análisis predictivo con regresión generalizada (GLM) - MODELO 3	49
Análisis predictivo con redes neuronales parametría 1 - MODELO 4	50
Análisis predictivo con redes neuronales parametría 2 - MODELO 5	52
Análisis predictivo con redes neuronales parametría 3 - MODELO 6	53
Análisis predictivo random forest - MODELO 7	54
Análisis predictivo xgboost - MODELO 8	56
Análisis predictivo xgboost no escalado - MODELO 8B	57
RESUMEN DE RESULTADOS	59
Predicción de bancos nuevos para dataset de radios censales sin bancos	59

OVERPASS TURBO

Comercios generales

```
[out:json][timeout:120];

area["name"]="Ciudad Autónoma de Buenos Aires"->.searchArea;

(
  // Comercios generales

  node["shop"](area.searchArea);
  way["shop"](area.searchArea);
  relation["shop"](area.searchArea);
);

out center;

=====
```

Gastronomía

```
[out:json][timeout:120];

area["name"]="Ciudad Autónoma de Buenos Aires"->.searchArea;

(
  // Gastronomía

  node["amenity"]="restaurant"(area.searchArea);
  node["amenity"]="cafe"(area.searchArea);
  node["amenity"]="bar"(area.searchArea);
  way["amenity"]="restaurant"(area.searchArea);
  way["amenity"]="cafe"(area.searchArea);
  way["amenity"]="bar"(area.searchArea);
);

out center;

=====
```

Mercados y supermercados

```
[out:json][timeout:120];

area["name"]="Ciudad Autónoma de Buenos Aires"->.searchArea;

(
  // Mercados y supermercados

  node["amenity"]="marketplace"(area.searchArea);
  way["amenity"]="marketplace"(area.searchArea);
);

out center;
```

=====

Hoteles y hospedajes

```
[out:json][timeout:120];

area["name"]="Ciudad Autónoma de Buenos Aires"->.searchArea;

(

    // Hoteles y hospedajes

    node["tourism"]="hotel" (area.searchArea);

    way["tourism"]="hotel" (area.searchArea);

);

out center;
```

=====

Industria

```
[out:json][timeout:120];

area["name"]="Ciudad Autónoma de Buenos Aires"->.searchArea;

(

    // Industria

    node["landuse"]="industrial" (area.searchArea);

    way["landuse"]="industrial" (area.searchArea);

    node["industrial"] (area.searchArea);

    way["industrial"] (area.searchArea);

);

out center;
```

=====

Salud y Transporte

```
[out:json][timeout:120];

area["name"]="Ciudad Autónoma de Buenos Aires"->.searchArea;

(

    // Salud y Transporte

    node["healthcare"]="hospital" (area.searchArea);

    way["healthcare"]="hospital" (area.searchArea);

    node["aeroway"]="gate" (area.searchArea);

    way["aeroway"]="gate" (area.searchArea);

    node["railway"]="station" (area.searchArea);

    way["railway"]="station" (area.searchArea);

);

out center;
```


POSTGIS

Extensión postgis

```
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE EXTENSION IF NOT EXISTS postgis_topology;

SELECT f_table_schema, f_table_name, f_geometry_column, srid, type
FROM public.geometry_columns;
```

Tabla de capas point por radio censal

```
-- Creación nueva tabla para capas point

CREATE TABLE "Resumen_por_cro_point" AS

SELECT  r.cro,

        COALESCE(cg.cantidad, 0) AS "Comercios Generales",
        COALESCE(dd.cantidad, 0) AS "Depósitos Distribuidoras",
        COALESCE(es.cantidad, 0) AS "Estaciones de Servicio",
        COALESCE(ga.cantidad, 0) AS "Gastronomía",
        COALESCE(hh.cantidad, 0) AS "Hoteles y Hospedajes",
        COALESCE(ind.cantidad, 0) AS "Industria",
        COALESCE(ic.cantidad, 0) AS "Inmobiliarias y Constructoras",
        COALESCE(le.cantidad, 0) AS "Locales para Esparcimiento",
        COALESCE(ms.cantidad, 0) AS "Mercados y Supermercados",
        COALESCE(ps.cantidad, 0) AS "Pago de Servicios",
        COALESCE(sa.cantidad, 0) AS "Salud y Transporte",
        COALESCE(ba.cantidad, 0) AS "Bancos"

FROM "Radios Censales OK" r

LEFT JOIN (

    SELECT rc.cro, COUNT(*) AS cantidad

    FROM "Radios Censales OK" rc

    JOIN "Bancos" p ON ST_Contains(rc.geom, p.geom)

    GROUP BY rc.cro

) ba ON r.cro = ba.cro

LEFT JOIN (

    SELECT rc.cro, COUNT(*) AS cantidad

    FROM "Radios Censales OK" rc

    JOIN "Comercios Generales" p ON ST_Contains(rc.geom, p.geom)

    GROUP BY rc.cro
```

```

) cg ON r.cro = cg.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Depósitos Distribuidoras" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) dd ON r.cro = dd.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Estaciones de Servicio" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) es ON r.cro = es.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Gastronomía" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) ga ON r.cro = ga.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Hoteles y Hospedajes" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) hh ON r.cro = hh.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Industria" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) ind ON r.cro = ind.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Inmobiliarias y Constructoras" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro

```

```

) ic ON r.cro = ic.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Locales para Esparcimiento" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) le ON r.cro = le.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Mercados y Supermercados" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) ms ON r.cro = ms.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Pago de Servicios" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) ps ON r.cro = ps.cro
LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Salud y Transporte" p ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) sa ON r.cro = sa.cro
;

```

```

-- Consulta tabla resumen por cro capas punto

```

```

SELECT *
FROM "Resumen_por_cro_point"
LIMIT 3;

```

```

-- Se cambia el tipo de dato para cro a texto

```

```

ALTER TABLE "Resumen_por_cro_point"
ALTER COLUMN cro TYPE text
USING cro::text;

```

Tabla de capas polygon por radio censal

```
-- Creación tabla resumen por cro capas poligonales

CREATE TABLE "Resumen_por_cro_polyg" AS

SELECT

    cm.cro,

    cm.c1 AS "Calidad de Materiales 1",

    cm.c2 AS "Calidad de Materiales 2",

    cm.c3 AS "Calidad de Materiales 3",

    cm.c4 AS "Calidad de Materiales 4",

    cm.ignorado AS "Calidad de Materiales no informado",

    h.hog AS "Hogares",

    h.vp AS "Viviendas"

FROM "Calidad de Materiales" cm

JOIN "Hogares" h

    ON cm.cro = h.cro;


-- Consulta tabla resumen por cro capas poligonales

SELECT *

FROM "Resumen_por_cro_polyg"

LIMIT 3;


-- Se cambia el tipo de dato para cro a texto

ALTER TABLE "Resumen_por_cro_polyg"

ALTER COLUMN cro TYPE text

USING cro::text;
```

Tabla de información estadística por radio censal

```
-- Crear tabla para datos censales

CREATE TABLE "Resumen_por_cro_censo" (

    cro TEXT,

    "Porcentaje Propietarios" DOUBLE PRECISION,

    "Porcentaje con Acceso a Internet" DOUBLE PRECISION,

    "Porcentaje NBI" DOUBLE PRECISION,

    "Habitantes" INTEGER,

    "Sin instrucción" INTEGER,
```

```

        "Primario Incompleto" INTEGER,
        "Primario Completo" INTEGER,
"Secundario Incompleto" INTEGER,
        "Secundario Completo" INTEGER,
        "Terciario Incompleto" INTEGER,
        "Terciario Completo" INTEGER,
        "Universitario Incompleto" INTEGER,
"Universitario Completo" INTEGER,
        "Posgrado Incompleto" INTEGER,
"Posgrado Completo" INTEGER,
        "OS / Prepaga" INTEGER,
        "Cobertura Estatal" INTEGER,
        "Sin Cobertura" INTEGER,
"Ocupado" INTEGER,
        "Desocupado" INTEGER,
        "Inactivo" INTEGER
);

```

```

-- Importar datos censales
COPY "Resumen_por_cro_censo"
FROM 'C:\Resumen_por_cro_censo.csv'
WITH (
    FORMAT CSV,
    HEADER TRUE,
    DELIMITER ',',
    ENCODING 'UTF8'
);

```

```

-- Consulta tabla resumen por cro datos censo
SELECT *
FROM "Resumen_por_cro_censo"
LIMIT 3;

```

Tabla integral de capas point, poligonales y datos estadísticos por radio censal

```

-- Crear tabla única de datos desde polyg, point y censo
CREATE TABLE "Resumen_por_cro" AS
SELECT *
FROM "Resumen_por_cro_censo" AS censo
LEFT JOIN "Resumen_por_cro_point" AS point USING (cro)
LEFT JOIN "Resumen_por_cro_polyg" AS polyg USING (cro);

-- Consulta tabla resumen por cro
SELECT *
FROM "Resumen_por_cro"
LIMIT 3;

-- Ver listado de columnas
SELECT
    column_name,
    data_type
FROM information_schema.columns
WHERE table_name = 'Resumen_por_cro';

SELECT COUNT(*) FROM "Resumen_por_cro";

```

Test variable nueva: tabla de capas puntuales + radio de cercanía 500 m por radio censal

```

-- VARIABLE DATOS PUNTUALES POR CRO EN CENTROIDE + 500 METROS
-- Creación nueva tabla datos puntuales ampliados
-- Parámetro de distancia (metros)
CREATE TABLE "Resumen_por_cro_point_ampl" AS
SELECT
    r.cro,

    COALESCE(bancos.cnt, 0) AS "Bancos",
    COALESCE(comercios_generales.cnt, 0) AS "Comercios Generales",
    COALESCE(depositos_distribuidoras.cnt, 0) AS "Depósitos Distribuidoras",
    COALESCE(ess.cnt, 0) AS "Estaciones de Servicio",
    COALESCE(gastro.cnt, 0) AS "Gastronomía",
    COALESCE(hoteles.cnt, 0) AS "Hoteles y Hospedajes",

```

```

        COALESCE(industria.cnt, 0)                AS "Industria",
        COALESCE(inmobiliarias.cnt, 0)            AS "Inmobiliarias y
Constructoras",
        COALESCE(ocio.cnt, 0)                    AS "Locales para
Esparcimiento",
        COALESCE(mercados.cnt, 0)                AS "Mercados y Supermercados",
        COALESCE(pagos.cnt, 0)                   AS "Pago de Servicios",
        COALESCE(transporte.cnt, 0)              AS "Salud y Transporte"

FROM "Radios Censales OK" AS r
-- Bancos
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Bancos" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS bancos ON TRUE
-- Comercios Generales
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Comercios Generales" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS comercios_generales ON TRUE

-- Depósitos Distribuidoras
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Depósitos Distribuidoras" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS depositos_distribuidoras ON TRUE
-- Estaciones de Servicio
LEFT JOIN LATERAL (

```

```

SELECT COUNT(*) AS cnt
FROM "Estaciones de Servicio" pt
WHERE
    ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
    AND NOT ST_Covers(r.geom, pt.geom)
) AS ess ON TRUE
-- Gastronomía
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Gastronomía" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS gastro ON TRUE

-- Hoteles y Hospedajes
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Hoteles y Hospedajes" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS hoteles ON TRUE
-- Industria
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Industria" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS industria ON TRUE
-- Inmobiliarias y Constructoras
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Inmobiliarias y Constructoras" pt
    WHERE

```

```

        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
    ) AS inmobiliarias ON TRUE
-- Locales para Esparcimiento
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Locales para Esparcimiento" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS ocio ON TRUE
-- Mercados y Supermercados
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Mercados y Supermercados" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS mercados ON TRUE
-- Pago de Servicios
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Pago de Servicios" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS pagos ON TRUE
-- Salud y Transporte
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Salud y Transporte" pt
    WHERE
        ST_DWithin(pt.geom::geography, ST_Centroid(r.geom)::geography, 500)
        AND NOT ST_Covers(r.geom, pt.geom)
) AS transporte ON TRUE
;

```

```
-- Consulta tabla resumen por cro
SELECT *
FROM "Resumen_por_cro_point_ampl"
LIMIT 3;
```

```
-- Los valores no reflejan correctamente el concepto de cercanía
-- Hay varios cro con formas y extensiones que no permiten calcular
correctamente los puntos cercanos
-- Se prueba con un búfer de 200 metros
```

Test variable nueva: tabla de capas puntuales + búfer de cercanía 200 m por radio censal

```
-- VARIABLE PUNTOS FUERA DE POLÍGONO EN BÚFER 200 METROS

-- 1) Precalcular el ANILLO exterior de 200 m por CRO, SIN cambiar SRID de
origen
--      (Buffer geodésico en metros sobre geography, restado el polígono
original)
DROP MATERIALIZED VIEW IF EXISTS radios_outer_ring_200m_geo;
CREATE MATERIALIZED VIEW radios_outer_ring_200m_geo AS
SELECT
    r.cro,
    r.geom, -- MultiPolygon en SRID 4326
    -- Anillo: buffer geodésico de 200 m MENOS el polígono original
    ST_Difference(
        ST_Buffer(r.geom::geography, 200)::geometry, -- vuelve a geometry
        (SRID 4326)
        r.geom
    ) AS ring200
FROM "Radios Censales OK" r;

-- Índices para acelerar los intersects posteriores
CREATE INDEX IF NOT EXISTS radios_outer_ring_200m_geo_ring_gix ON
radios_outer_ring_200m_geo USING GIST (ring200);
CREATE INDEX IF NOT EXISTS radios_outer_ring_200m_geo_cro_idx ON
radios_outer_ring_200m_geo (cro);

-- 2) Crear la tabla de RESUMEN: una fila por CRO, una columna por capa
```

```

CREATE TABLE "Resumen_por_cro_point_ampl2" AS
SELECT
    rr.cro,

    COALESCE(bancos.cnt, 0) AS "Bancos",
    COALESCE(comercios_generales.cnt, 0) AS "Comercios Generales",
    COALESCE(depositos_distribuidoras.cnt, 0) AS "Depósitos Distribuidoras",
    COALESCE(ess.cnt, 0) AS "Estaciones de Servicio",
    COALESCE(gastro.cnt, 0) AS "Gastronomía",
    COALESCE(hoteles.cnt, 0) AS "Hoteles y Hospedajes",
    COALESCE(industria.cnt, 0) AS "Industria",
    COALESCE(inmobiliarias.cnt, 0) AS "Inmobiliarias y
Constructoras",
    COALESCE(ocio.cnt, 0) AS "Locales para
Esparcimiento",
    COALESCE(mercados.cnt, 0) AS "Mercados y Supermercados",
    COALESCE(pagos.cnt, 0) AS "Pago de Servicios",
    COALESCE(transporte.cnt, 0) AS "Salud y Transporte"

FROM radios_outer_ring_200m_geo rr

-- Bancos
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Bancos" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS bancos ON TRUE

-- Comercios Generales
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Comercios Generales" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS comercios_generales ON TRUE

-- Depósitos Distribuidoras
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Depósitos Distribuidoras" p

```

```

WHERE ST_Intersects(p.geom, rr.ring200)
) AS depositos_distribuidoras ON TRUE
-- Estaciones de Servicio
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Estaciones de Servicio" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS ess ON TRUE
-- Gastronomía
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Gastronomía" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS gastro ON TRUE
-- Hoteles y Hospedajes
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Hoteles y Hospedajes" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS hoteles ON TRUE
-- Industria
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Industria" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS industria ON TRUE
-- Inmobiliarias y Constructoras
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Inmobiliarias y Constructoras" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS inmobiliarias ON TRUE
-- Locales para Esparcimiento
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Locales para Esparcimiento" p

```

```

        WHERE ST_Intersects(p.geom, rr.ring200)
    ) AS ocio ON TRUE
-- Mercados y Supermercados
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Mercados y Supermercados" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS mercados ON TRUE
-- Pago de Servicios
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Pago de Servicios" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS pagos ON TRUE
-- Salud y Transporte
LEFT JOIN LATERAL (
    SELECT COUNT(*) AS cnt
    FROM "Salud y Transporte" p
    WHERE ST_Intersects(p.geom, rr.ring200)
) AS transporte ON TRUE
;

-- Consulta tabla resumen por cro
SELECT *
FROM "Resumen_por_cro_point_ampl2"
LIMIT 3;

-- Se cambia el tipo de dato para cro a texto
ALTER TABLE "Resumen_por_cro_point_ampl2"
ALTER COLUMN cro TYPE text
USING cro::text;

--Creación de tabla ampliada de datos puntuales + bufer
CREATE TABLE "Resumen_por_cro_point_bufer" AS
SELECT
    COALESCE(a."cro", b."cro") AS "cro",

```

```

        COALESCE(a."Comercios Generales", 0) + COALESCE(b."Comercios Generales",
0) AS "Comercios Generales_2",

        COALESCE(a."Depósitos Distribuidoras", 0) + COALESCE(b."Depósitos
Distribuidoras", 0) AS "Depósitos Distribuidoras_2",

        COALESCE(a."Estaciones de Servicio", 0) + COALESCE(b."Estaciones de
Servicio", 0) AS "Estaciones de Servicio_2",

        COALESCE(a."Gastronomía", 0) + COALESCE(b."Gastronomía", 0) AS
"Gastronomía_2",

        COALESCE(a."Hoteles y Hospedajes", 0) + COALESCE(b."Hoteles y
Hospedajes", 0) AS "Hoteles y Hospedajes_2",

        COALESCE(a."Industria", 0) + COALESCE(b."Industria", 0) AS
"Industria_2",

        COALESCE(a."Inmobiliarias y Constructoras", 0) +
COALESCE(b."Inmobiliarias y Constructoras", 0) AS "Inmobiliarias y
Constructoras_2",

        COALESCE(a."Locales para Esparcimiento", 0) + COALESCE(b."Locales para
Esparcimiento", 0) AS "Locales para Esparcimiento_2",

        COALESCE(a."Mercados y Supermercados", 0) + COALESCE(b."Mercados y
Supermercados", 0) AS "Mercados y Supermercados_2",

        COALESCE(a."Pago de Servicios", 0) + COALESCE(b."Pago de Servicios", 0)
AS "Pago de Servicios_2",

        COALESCE(a."Salud y Transporte", 0) + COALESCE(b."Salud y Transporte",
0) AS "Salud y Transporte_2",

        COALESCE(a."Bancos", 0) + COALESCE(b."Bancos", 0) AS "Bancos_2"

FROM "Resumen_por_cro_point" a
FULL OUTER JOIN "Resumen_por_cro_point_ampl2" b
    ON A."cro"= b."cro"
ORDER BY "cro";

;

-- Consulta tabla resumen por cro
SELECT *
FROM "Resumen_por_cro_point_bufer"
LIMIT 3;

```

Test variable nueva: tabla de cantidad de metros de avenida por radio censal

```
-- VARIABLE CANTIDAD DE METROS POR CRO
```

```

-- Creación tabla metros de avenida por cro
CREATE TABLE "Resumen_por_cro_ave" AS
SELECT
    r.cro,
    COALESCE(SUM(ST_Length(ST_Intersection(r.geom::geography,
a.geom::geography))), 0) AS metros_avenida
FROM "Radios Censales OK" AS r
LEFT JOIN "Avenidas" AS a
    ON ST_Intersects(r.geom, a.geom)
GROUP BY r.cro
ORDER BY r.cro;

-- Usar solo fracción entera de las cifras
UPDATE "Resumen_por_cro_ave"
SET metros_avenida = TRUNC (CAST(metros_avenida AS numeric));

-- Consulta tabla resumen por cro
SELECT *
FROM "Resumen_por_cro_ave"
LIMIT 3;

-- Se cambia el tipo de dato para cro a texto
ALTER TABLE "Resumen_por_cro_ave"
ALTER COLUMN cro TYPE text
USING cro::text;

```

Test variable nueva: tabla de cantidad de bancos sobre avenidas por radio censal + búfer de 500m

```

-- VARIABLE CANTIDAD DE BANCOS EN CRO + BÚFER 500 METROS

-- Crear vista con bancos sobre búfer de avenidas
DROP MATERIALIZED VIEW IF EXISTS mv_ave_buf500_por_radio;

CREATE MATERIALIZED VIEW mv_ave_buf500_por_radio AS
SELECT
    r.cro,

```

-- Segmento de avenida dentro del radio → buffer 500 m (geodesic) → unión por CRO

```
ST_UnaryUnion(
  ST_Collect(
    ST_Buffer(
      ST_Intersection(a.geom, r.geom)::geography, 500
    )::geometry
  )
) AS buf500
FROM "Radios Censales OK" r
JOIN "Avenidas" a
  ON ST_Intersects(r.geom, a.geom)      -- solo avenidas que cruzan el
radio
GROUP BY r.cro;
```

```
CREATE INDEX IF NOT EXISTS mv_ave_buf500_por_radio_gix ON
mv_ave_buf500_por_radio USING GIST (buf500);
```

```
CREATE INDEX IF NOT EXISTS mv_ave_buf500_por_radio_cro ON
mv_ave_buf500_por_radio (cro);
```

-- Creación de tabla resumen por cro + búfer 500 metros

```
CREATE TABLE "Resumen_por_cro_ave_ban" AS
SELECT
  r.cro,
  COALESCE(b.cant_bancos, 0) AS cant_bancos_500m_avenidas
FROM "Radios Censales OK" r
LEFT JOIN mv_ave_buf500_por_radio m USING (cro)
LEFT JOIN LATERAL (
  SELECT COUNT(*) AS cant_bancos
  FROM "Bancos" pts
  WHERE m.buf500 IS NOT NULL
    AND ST_Intersects(pts.geom, m.buf500)
) AS b ON TRUE;
```

-- Consulta tabla resumen por cro

```
SELECT *
FROM "Resumen_por_cro_ave_ban"
LIMIT 3;
```

```
-- Se cambia el tipo de dato para cro a texto
ALTER TABLE "Resumen_por_cro_ave_ban"
ALTER COLUMN cro TYPE text
USING cro::text;
```

Test variable nueva: tabla de cantidad de bancos BNA por radio censal

```
-- Creación nueva tabla para capas point BNA
CREATE TABLE "Resumen_por_cro_bna" AS
SELECT
    r.cro,
    COALESCE(bna.cantidad, 0) AS "Banco de la Nacion Argentina"

FROM "Radios Censales OK" r

LEFT JOIN (
    SELECT rc.cro, COUNT(*) AS cantidad
    FROM "Radios Censales OK" rc
    JOIN "Banco de la Nación Argentina" p
    ON ST_Contains(rc.geom, p.geom)
    GROUP BY rc.cro
) bna ON r.cro = bna.cro
;

-- Consulta tabla resumen por cro capas punto
SELECT *
FROM "Resumen_por_cro_bna"
LIMIT 3;

-- Se cambia el tipo de dato para cro a texto
ALTER TABLE "Resumen_por_cro_bna"
ALTER COLUMN cro TYPE text
USING cro::text;
```

Tabla integral dataset final

```
-- TABLA CON DATASET FINAL

-- Tabla de Radios Censales con valor cro texto y polígono
CREATE TABLE "RC Multipolígono" AS
SELECT
    ("cro")::text AS "cro",
    ST_Multi(geom) AS geom
FROM "Radios Censales OK";

-- Consulta tabla resumen por cro capas punto
SELECT *
FROM "RC Multipolígono"
LIMIT 3;

-- Creación tabla "Resumen"
-- Unifica por cro todas las tablas indicadas + geometría de RC
Multipolígono
CREATE TABLE "Resumen" AS
WITH base AS (
    SELECT "cro" FROM "Resumen_por_cro_bna"
    UNION
    SELECT "cro" FROM "Resumen_por_cro_point"
    UNION
    SELECT "cro" FROM "Resumen_por_cro_point_bufer"
    UNION
    SELECT "cro" FROM "Resumen_por_cro_polyg"
    UNION
    SELECT "cro" FROM "Resumen_por_cro_censo"
    UNION
    SELECT "cro" FROM "Resumen_por_cro_ave"
    UNION
    SELECT "cro" FROM "Resumen_por_cro_ave_ban"
)
SELECT *
FROM base
LEFT JOIN "RC Multipolígono"          USING ("cro")  -- aporta geom
(geometry)
```

```

LEFT JOIN "Resumen_por_cro_bna"      USING ("cro")
LEFT JOIN "Resumen_por_cro_point"    USING ("cro")
LEFT JOIN "Resumen_por_cro_point_bufer" USING ("cro")
LEFT JOIN "Resumen_por_cro_polyg"    USING ("cro")
LEFT JOIN "Resumen_por_cro_censo"     USING ("cro")
LEFT JOIN "Resumen_por_cro_ave"       USING ("cro")
LEFT JOIN "Resumen_por_cro_ave_ban"   USING ("cro")
ORDER BY "cro";

```

```
-- Consulta tabla RESUMEN
```

```

SELECT *
FROM "Resumen"
LIMIT 3;

```

Exportación de tabla resumen

```
-- Extracción de tabla RESUMEN
```

```

COPY "Resumen" TO 'C:/Users/claud/Resumen.csv' WITH CSV HEADER DELIMITER
',' ENCODING 'UTF8';

```

Importación de tabla predicciones

```
-- Importación de tabla PREDICCIONES
```

```

CREATE TABLE "Predicciones" (
    id SERIAL PRIMARY KEY,
    cro TEXT,
    ban_nvo integer
);

```

Modificación de tabla predicciones a datos geográficos

```
-- Agrego la columna geom
```

```

ALTER TABLE public."Predicciones"
ADD COLUMN geom geometry;

```

```
-- Extrae de Resumen los datos geom para agregarlos a Predicciones
```

```
UPDATE public."Predicciones" p
```

```
SET geom = r.geom  
FROM public."Resumen" r  
WHERE p.cro = r.cro;
```

```
-- Consulta tabla PREDICCIONES  
SELECT *  
FROM "Predicciones"  
LIMIT 3;
```

R

Preparación de ambiente

```
rm(list = ls())  
traceback()  
rm()
```

Carga de paquetes y librerías

```
install.packages("corrplot")  
install.packages("tidyverse")  
install.packages("car")  
install.packages(c("neuralnet", "NeuralNetTools", "dplyr", "caret"))  
install.packages("randomForest")  
install.packages("xgboost")  
  
library(ggplot2)  
library(dplyr)  
library(ggcorrplot)  
library(funModeling)  
library(readr)  
library(tidyverse)  
library(car)  
library(ggplot2)  
library(neuralnet)  
library(NeuralNetTools)  
library(caret)  
library(randomForest)  
library(xgboost)
```

Carga de dataset local

```
dataset_completo <- read_csv("C:/Users/claud/Documents/ITBA/11.  
TFI/Datos/Resumen.csv")
```

Exploración y modificación dataset

```
dim(dataset_completo)  
nrow(dataset_completo)  
ncol(dataset_completo)  
head(dataset_completo)  
names(dataset_completo)  
summary(dataset_completo)  
sapply(dataset_completo, class) # Características del dataset  
  
filasvacias <- subset(dataset_completo,  
!complete.cases(dataset_completo$cro))  
filasvacias # Revisión filas vacías
```

```

dataset <- dataset_completo %>%
  rename(
    BNA = 'Banco de la Nacion Argentina',
    CGE = 'Comercios Generales',
    DYD = 'Depósitos Distribuidoras',
    EDS = 'Estaciones de Servicio',
    GAS = 'Gastronomía',
    HYH = 'Hoteles y Hospedajes',
    IND = 'Industria',
    IYC = 'Inmobiliarias y Constructoras',
    LPE = 'Locales para Esparcimiento',
    MYS = 'Mercados y Supermercados',
    PDS = 'Pago de Servicios',
    SYT = 'Salud y Transporte',
    BAN = 'Bancos',
    CGE2 = 'Comercios Generales_2',
    DYD2 = 'Depósitos Distribuidoras_2',
    EDS2 = 'Estaciones de Servicio_2',
    GAS2 = 'Gastronomía_2',
    HYH2 = 'Hoteles y Hospedajes_2',
    IND2 = 'Industria_2',
    IYC2 = 'Inmobiliarias y Constructoras_2',
    LPE2 = 'Locales para Esparcimiento_2',
    MYS2 = 'Mercados y Supermercados_2',
    PDS2 = 'Pago de Servicios_2',
    SYT2 = 'Salud y Transporte_2',
    BAN2 = 'Bancos_2',
    CMA1 = 'Calidad de Materiales 1',
    CMA2 = 'Calidad de Materiales 2',
    CMA3 = 'Calidad de Materiales 3',
    CMA4 = 'Calidad de Materiales 4',
    CMANO = 'Calidad de Materiales no informado',
    HOG = 'Hogares',
    VIV = 'Viviendas',
    PPR = 'Porcentaje Propietarios',
    PAI = 'Porcentaje con Acceso a Internet',
    PNBI = 'Porcentaje NBI',
    HAB = 'Habitantes',
    SII = 'Sin instrucción',
    PIN = 'Primario Incompleto',
    PCO = 'Primario Completo',
    SIN = 'Secundario Incompleto',
    SCO = 'Secundario Completo',
    TIN = 'Terciario Incompleto',
    TCO = 'Terciario Completo',
    UIN = 'Universitario Incompleto',
    UCO = 'Universitario Completo',
    PSI = 'Posgrado Incompleto',
    PSC = 'Posgrado Completo',
    OSP = 'OS / Prepaga',
    COBE = 'Cobertura Estatal',
    SCOB = 'Sin Cobertura',
    OCU = 'Ocupado',
    DES = 'Desocupado',
    INA = 'Inactivo',
    MSAV = 'metros_avenida',
    CBA = 'cant_bancos_500m_avenidas'
  ) # Se achican los nombres de columnas para facilitar lectura

```

Gráficos boxplot completos por fuente de información

```
x11()
boxplot (dataset [,-1],
        col= "blue",
        main = "Gráficos Boxplot Total Muestra",
        xaxt = "n") # Oculta los labels originales del eje x
axis(1,
     labels = colnames(dataset[ , -1]),
     at = 1:ncol(dataset[ , -1]),
     las = 2,
     cex.axis = 0.6)

x11()
boxplot(dataset [ ,c(2,3,4,5,6,7,8,9,10,11,12,13,14)],
        col = 'brown1',
        main="Gráficos Boxplot Muestra Geog Point",
        cex.axis=0.8 )

x11()
boxplot(dataset [ ,c(15,16,17,18,19,20,21,22,23,24,25,26)],
        col = 'burlywood1',
        main="Gráficos Boxplot Muestra Geog Point Búfer",
        cex.axis=0.8 )

x11()
boxplot(dataset [ ,c(27,28,29,30,31,32,33,34,35,36,37)],
        col = 'cadetblue1',
        main="Gráficos Boxplot Muestra Polyg",
        cex.axis = 0.8 )

x11()
boxplot(dataset [ ,c(38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,53,54)],
        col = 'aquamarine1',
        main="Gráficos Boxplot Muestra Censo",
        cex.axis = 0.8 )

x11()
boxplot(dataset [ ,c(55,56)],
        col = 'pink',
        main="Gráficos Boxplot Avenidas",
        cex.axis = 0.8 )
```

Estadísticas generales

```
tabla_resumen <- dataset %>%
  select(-1) %>%
  summarise(across(
    everything(),
    list(
      media = ~mean(.x, na.rm = TRUE),
      q1 = ~quantile(.x, 0.25, na.rm = TRUE),
      q3 = ~quantile(.x, 0.75, na.rm = TRUE),
```

```

      min = ~min(.x, na.rm = TRUE),
      max = ~max(.x, na.rm = TRUE)
    ),
    .names = "{.col}_{.fn}"
  )) %>%
pivot_longer(
  everything(),
  names_to = c("variable", "estadístico"),
  names_sep = "_",
  values_to = "valor"
) %>%
pivot_wider(
  names_from = estadístico,
  values_from = valor
)

# --> CONCLUSIÓN: metros de avenida por cro es el valor más elevado

outlier_counts <- map_dfr(dataset, function(x) {
  q1 <- quantile(x, 0.25, na.rm = TRUE)
  q3 <- quantile(x, 0.75, na.rm = TRUE)
  iqr <- q3 - q1
  outliers <- sum(x < (q1 - 1.5 * iqr) | x > (q3 + 1.5 * iqr), na.rm =
TRUE)
  tibble(outliers = outliers)
}, .id = "variable")

outlier_counts

# --> CONCLUSIÓN: Mercados y supermercados es la variable con más outliers

tabla_resumen <- tabla_resumen %>%
  left_join(outlier_counts, by = "variable")

tabla_resumen <- tabla_resumen %>% arrange(desc(media))
print(tabla_resumen, n = Inf)

```

Gráficos boxplot ordenados por intervalos

```

names (dataset)

x11()
boxplot(dataset [,c("HAB", "OSP")],
  col = 'firebrick1',
  main="Gráficos Boxplot Valores mayores",
  cex.axis=0.8 )

x11()
boxplot(dataset [,c("OCU", "VIV", "HOG", "CMA1", "INA", "MSAV", "SCO",
"UIN", "SIN", "UCO")],
  col = 'hotpink1',
  main="Gráficos Boxplot Valores intermedios altos",
  cex.axis=0.8,
  ylim=c(0,1500))

x11()

```

```

boxplot(dataset
[,c("PAI", "PIN", "TCO", "SII", "PPR", "CMA2", "CGE2", "PCO", "TIN", "PSC", "CMANO", "DES")],
      col = 'indianred1',
      main="Gráficos Boxplot Valores intermedios",
      cex.axis=0.8,
      ylim=c (0,400))

x11()
boxplot(dataset [,c("PSI", "DYD2", "COBE", "GAS2", "CMA3", "IYC2", "PNBI")],
      col = 'green2',
      main="Gráficos Boxplot Valores intermedios bajos",
      cex.axis=0.8,
      ylim=c (0,150))

x11()
boxplot(dataset
[,c("CGE", "CBA", "MYS2", "DYD", "BAN2", "LPE2", "PDS2", "CMA4", "HYH2")],
      col = 'gold2',
      main="Gráficos Boxplot Valores bajos",
      cex.axis=0.8 ,
      ylim=c(0,40))

x11()
boxplot(dataset
[,c("SYT2", "IYC", "EDS2", "MYS", "BAN", "IND2", "LPE", "PDS", "HYH", "SYT", "EDS", "IND", "BNA")],
      col = 'goldenrod3',
      main="Gráficos Boxplot Valores muy bajos",
      cex.axis=0.8 ,
      ylim = c(0, 5))

# --> 6) Análisis datos columna objetivo BAN2

# 1.- Agrupar BAN2
resumen_BAN2 <- dataset %>%
  group_by(BAN2) %>%
  summarise(cantidad = n()) %>%
  arrange(BAN2)

resumen_BAN2

# --> El mayor número de bancos por cro es 92
# --> Hay 1792 cro sin bancos (aprox el 50%)

x11()
ggplot(
  data = subset(resumen_BAN2, BAN2 > 0),
  aes(x = cantidad, y = BAN2)
) +
  geom_point(color = "steelblue", size = 3, alpha = 0.7) + # puntos
  originales
  geom_line(color = "gold2", linewidth = 1, alpha = 0.5) + # une los
  puntos
  scale_x_reverse() + # eje X
  invertido
  theme_minimal() +
  labs(
    title = "Cantidad de CRO por valor de BAN2",
    x = "Cantidad de CRO con el mismo BAN2",

```

```
y = "Valor de BAN2"  
)
```

Análisis de correlaciones

```
datos_cor <- dataset[, -c(1,2)]  
head(datos_cor)
```

```
matriz_cor <- cor(datos_cor)  
datos_cor  
round(matriz_cor[1:10, 1:10], 2)
```

```
x11()  
ggcorrplot(matriz_cor,  
            tl.cex=6,  
            colors = c("blue", "white", "firebrick1"),  
            title="Matriz de correlaciones")
```

```
x11()  
# Graficar la matriz ordenada  
ggcorrplot(matriz_cor,  
            method = "square",      # estilo del gráfico  
            type = "lower",         # muestra solo mitad inferior  
            lab_size = 2.5,         # tamaño de texto  
            hc.order = TRUE,        # ordena por clustering jerárquico  
            tl.cex = 6,             # tamaño de etiquetas  
            tl.col = "black",       # color del texto  
            title = "Matriz de correlaciones ordenada",  
            ggtheme = ggplot2::theme_minimal)
```

```
# Variables más correlacionadas  
cor_data <- as.data.frame(as.table(matriz_cor)) # 1) Pasar la matriz a formato largo  
cor_data <- cor_data[cor_data$Var1 != cor_data$Var2, ] # 2) Eliminar la diagonal (correlaciones  
de una variable consigo misma)  
cor_data_ord <- cor_data[order(-abs(cor_data$Freq)), ] # 3) Ordenar por correlación (valor  
absoluto)  
head(cor_data_ord, 30) # 4) Mostrar las más correlacionadas  
  
cor_data_fuerte <- subset(cor_data_ord, Freq > 0.85 | Freq < -0.85)  
cor_data_fuerte
```

```

# matriz de correlaciones calculadas fuertes
umbral <- 0.85
vars_fuertes <- apply(matriz_cor, 2,
                      function(x) any(abs(x) > umbral & abs(x) < 1))
matriz_cor_filtrada <- matriz_cor[vars_fuertes, vars_fuertes]

x11()
ggcorrplot(matriz_cor_filtrada,
            method = "square",
            type = "lower",
            lab = TRUE,
            lab_size = 3.5,
            hc.order = TRUE,
            colors = c("blue", "white", "red"),
            title = paste("Correlaciones fuertes (>|", umbral, "|)", sep = ""),
            ggtheme = ggplot2::theme_minimal)

```

Se observa una correlación alta, lo ideal es que los valores estén más cerca de 1

```

det_cor <- det(matriz_cor)
det_cor

```

--> CONCLUSIÓN: -9.489753e-58: Alta multicolinealidad (variables casi linealmente dependientes)

```

shapiro.test(dataset$BAN2)

```

--> CONCLUSIÓN 2:

```

# W = 0.549 → muy lejos de 1, indica fuerte desviación de la normalidad.
# p < 0.05, se rechaza la hipótesis de normalidad.
# De acuerdo con el gráfico Cantidad de CRO por valor de BAN2 se verifica que la distribución
es extremadamente asimétrica (right-skewed):
# La mayoría de los valores BAN2 están cerca de 0.
# A medida que BAN2 aumenta, su frecuencia disminuye drásticamente.
# Existen valores altos de BAN2, pero son muy pocos (cola larga).
# La curva amarilla (tendencia suavizada) confirma una relación exponencial:
# Esto refuerza lo observado en el test Shapiro-Wilk: la distribución NO es normal.
# BAN2 está fuertemente concentrado en valores bajos y presenta algunos valores muy altos
con poca frecuencia, lo que indica una distribución altamente asimétrica y no normal, con cola
larga y posibles outliers reales.

```

Preparación de dataset para modelos

```
# Crear dataset para modelo: cro con o sin bancos --> Usar BAN2 y eliminar
BAN
dataset_bancos <- subset(dataset[, -c(2, 14)], BAN2 > 0)
dataset_sin_bancos <- subset(dataset[, -c(2, 14)], BAN2 == 0)
nrow(dataset_bancos)
nrow(dataset_sin_bancos)

dataset_bna <- subset(dataset, BNA > 0)

dataset
```

Análisis predictivo por regresión lineal simple - MODELO 1

```
# Creación de modelo 1
set.seed(314)
datos_modelo_1 <- dataset_bancos
train_index_1 <- sample(1:nrow(datos_modelo_1), 0.7 * nrow(datos_modelo_1))
#

# Crear subconjuntos
train_data_1 <- datos_modelo_1 [train_index_1, ]
test_data_1 <- datos_modelo_1 [-train_index_1, ]

modelo_1 <- lm(BAN2 ~ ., data = train_data_1)
summary(modelo_1)
predicciones_1 <- predict(modelo_1, newdata = test_data_1)
predicciones_1

# Evaluación modelo_1
rmse <- sqrt(mean((test_data_1$BAN2 - predicciones_1)^2)) # RMSE
mae <- mean(abs(test_data_1$BAN2 - predicciones_1)) # MAE
r2 <- 1 - sum((test_data_1$BAN2 - predicciones_1)^2) /
sum((test_data_1$BAN2 - mean(test_data_1$BAN2))^2) # R²
cat("RMSE:", rmse, "\n")
cat("MAE:", mae, "\n")
cat("R²:", r2, "\n")

x11()
plot(test_data_1$BAN2, predicciones_1,
     xlab = "Valores reales (BAN2)",
     ylab = "Valores predichos",
     main = "Comparación entre BAN2 real y predicho",
     pch = 19, col = "blue")
abline(0, 1, col = "red", lwd = 2) # línea perfecta
```

Análisis predictivo por regresión lineal simple con PCE - MODELO 2

```
# Cálculo del Variance Inflation Factor
vif_valores <- vif(modelo_1)
print(vif_valores)
# El cálculo informa que hay variables perfectamente correlacionadas

alias(modelo_1)
# Variables problemáticas:
#   - HOG: CMA1 CMA2 CMA3 CMA4 CAMNO
#   - SCOB: HAB OSP COBE

# Aplicación de PCA
predictores <- dataset_bancos[ , -c(24)] # selección de datos predictores
predictores_scaled <- scale(predictores) # se escalan los valores
pca_result <- prcomp(predictores_scaled, center = TRUE, scale. = TRUE)
summary(pca_result)

varianza <- pca_result$sdev^2 / sum(pca_result$sdev^2) # varianza por
componente
varianza
varianza_acum <- cumsum(varianza) # varianza acumulada
varianza_acum

num_comp <- which(varianza_acum >= 0.90)[1] # componentes necesarios para
explicar al menos el 90%
num_comp

# Nuevo dataset
componentes <- as.data.frame(pca_result$x[ , 1:num_comp]) # agregar las
comp necesarias para 90%
componentes$BAN2 <- dataset_bancos$BAN2 # agregar variable objetivo
componentes$cro <- dataset_bancos$cro
componentes <- componentes[ , c(ncol(componentes), 1:(ncol(componentes)-
1))]

# Creación de modelo 2
set.seed(315)
datos_modelo_2 <- componentes
train_index_2 <- sample(1:nrow(datos_modelo_2), 0.7 * nrow(datos_modelo_2))
#

# Crear subconjuntos
train_data_2 <- datos_modelo_2[train_index_2, ]
test_data_2 <- datos_modelo_2[-train_index_2, ]

modelo_2 <- lm(BAN2 ~ ., data = train_data_2)
summary(modelo_2)
predicciones_2 <- predict(modelo_2, newdata = test_data_2)
predicciones_2

# Evaluación modelo_2
rmse_2 <- sqrt(mean((test_data_2$BAN2 - predicciones_2)^2)) # RMSE
mae_2 <- mean(abs(test_data_2$BAN2 - predicciones_2)) # MAE
r2_2 <- 1 - sum((test_data_2$BAN2 - predicciones_2)^2) /
```

```

sum((test_data_2$BAN2 - mean(test_data_2$BAN2))^2) # R²
cat("RMSE:", rmse_2, "\n")
cat("MAE:", mae_2, "\n")
cat("R²:", r2_2, "\n")

x11()
plot(test_data_2$BAN, predicciones_2,
      xlab = "Valores reales (BAN)",
      ylab = "Valores predichos",
      main = "Comparación entre BAN real y predicho",
      pch = 19, col = "blue")
abline(0, 1, col = "red", lwd = 2) # línea perfecta

# ---> CONCLUSIÓN: Dada la naturaleza del dataset, los métodos lineales
# clásicos (regresión lineal, ANOVA, Pearson)
# NO van a funcionar bien, incluso con transformaciones.

```

Análisis predictivo con regresión generalizada (GLM) - MODELO 3

```

# Creación de modelo 3
set.seed(316)
datos_modelo_3 <- dataset_bancos
train_index_3 <- sample(1:nrow(datos_modelo_3), 0.7 * nrow(datos_modelo_3))
#

# Crear subconjuntos
train_data_3 <- datos_modelo_3[train_index_3, ]
test_data_3 <- datos_modelo_3[-train_index_3, ]

modelo_3 <- glm(BAN2 ~ ., data = train_data_3, family = quasipoisson) #para
BAN2 discreto
summary(modelo_3)
predicciones_3 <- predict(modelo_3, newdata = test_data_3)
predicciones_3

# Evaluación modelo_3
rmse_3 <- sqrt(mean((test_data_3$BAN2 - predicciones_3)^2)) # RMSE
mae_3 <- mean(abs(test_data_3$BAN2 - predicciones_3)) # MAE
r2_3 <- 1 - sum((test_data_3$BAN2 - predicciones_3)^2) /
sum((test_data_3$BAN2 - mean(test_data_3$BAN2))^2) # R²
cat("RMSE:", rmse_3, "\n")
cat("MAE:", mae_3, "\n")
cat("R²:", r2_3, "\n")

x11()
plot(test_data_3$BAN2, predicciones_3,
      xlab = "Valores reales (BAN2)",
      ylab = "Valores predichos",
      main = "Comparación entre BAN real y predicho",
      pch = 19, col = "blue")
abline(0, 1, col = "red", lwd = 2) # línea perfecta

```

Análisis predictivo con redes neuronales parametría 1 - MODELO 4

```
dataset_bancos_filtrado <- dataset_bancos %>%
  select(-cro)

preproc <- preProcess(dataset_bancos_filtrado, method = c("range"))
datos_scaled <- predict(preproc, dataset_bancos_filtrado)

str(datos_scaled)

set.seed(317)

n <- nrow(datos_scaled)
train_index_4 <- sample(1:n, size = 0.7 * n) # partición al 70%

train_data_4 <- datos_scaled[train_index_4, ]
test_data_4 <- datos_scaled[-train_index_4, ]

form <- as.formula(
  paste("BAN2 ~", paste(setdiff(names(datos_scaled), "BAN2"), collapse
= " + "))
)

modelo_4 <- neuralnet(
  formula = form,
  data = train_data_4,
  hidden = c(5, 3),
  linear.output = TRUE,
  lifesign = "minimal"
)

x11()
plotnet(modelo_4, cex_val = 0.6)

x11()
olden(modelo_4, cex=0.2)

predicciones_4 <- compute(
  modelo_4,
  test_data_4[, setdiff(names(test_data_4), "BAN2")]
)$net.result

y_real <- test_data_4$BAN2

mae_4 <- mean(abs(y_real - predicciones_4))
rmse_4 <- sqrt(mean((y_real - predicciones_4)^2))
r2_4 <- 1 - sum((y_real - predicciones_4)^2) / sum((y_real -
mean(y_real))^2)

cat("RMSE:", rmse_4, "\n")
cat("MAE :", mae_4, "\n")
cat("R² :", r2_4, "\n")
```

Análisis predictivo con redes neuronales parametría 2 - MODELO 5

```
set.seed(318)

n <- nrow(datos_scaled)
train_index_5 <- sample(1:n, size = 0.7 * n) # partición al 70%

train_data_5 <- datos_scaled[train_index_5, ]
test_data_5 <- datos_scaled[-train_index_5, ]

form <- as.formula(
  paste("BAN2 ~", paste(setdiff(names(datos_scaled), "BAN2"), collapse = "
+ "))
)

modelo_5 <- neuralnet(
  formula = form,
  data = train_data_5,
  hidden = c(10), # una capa: 10 neuronas (mayor
  capacidad para capturar no linealidades)
  linear.output = TRUE, # regresión
  lifesign = "minimal" # menos texto durante el
  entrenamiento
)

x11()
plotnet(modelo_5, cex_val = 0.6)

x11()
olden(modelo_5, cex=0.2)

predicciones_5 <- compute(
  modelo_5,
  test_data_5[, setdiff(names(test_data_5), "BAN2")]
)$net.result

y_real5 <- test_data_5$BAN2

mae_5 <- mean(abs(y_real5 - predicciones_5))
rmse_5 <- sqrt(mae_5)
r2_5 <- 1 - sum((y_real5 - predicciones_5)^2) / sum((y_real5 -
mean(y_real5))^2)

cat("RMSE:", rmse_5, "\n")
cat("MAE :", mae_5, "\n")
cat("R² :", r2_5, "\n")
```

Análisis predictivo con redes neuronales parametría 3 - MODELO 6

```
set.seed(319)
```

```

n <- nrow(datos_scaled)
train_index_6 <- sample(1:n, size = 0.7 * n) # partición al 70%

train_data_6 <- datos_scaled[train_index_6, ]
test_data_6 <- datos_scaled[-train_index_6, ]

form <- as.formula(
  paste("BAN2 ~", paste(setdiff(names(datos_scaled), "BAN2"), collapse = "
+ "))
)

modelo_6 <- neuralnet(
  formula = form,
  data = train_data_6,
  hidden = c(10,6,3),          # tres capas: con 10,6 y 3 neuronas
  (mayor capacidad de procesamiento)
  linear.output = TRUE,        # regresión
  lifesign = "minimal"        # menos texto durante el
  entrenamiento
)

x11()
plotnet(modelo_6, cex_val = 0.6)

x11()
olden(modelo_6, cex=0.2)

predicciones_6 <- compute(
  modelo_6,
  test_data_6[, setdiff(names(test_data_6), "BAN2")]
)$net.result

y_real6 <- test_data_6$BAN2

mae_6 <- mean(abs(y_real6 - predicciones_6))
rmse_6 <- sqrt(mae_6)
r2_6 <- 1 - sum((y_real6 - predicciones_6)^2) / sum((y_real6 -
mean(y_real6))^2)

cat("RMSE:", rmse_6, "\n")
cat("MAE :", mae_6, "\n")
cat("R²  :", r2_6, "\n")

```

Análisis predictivo random forest - MODELO 7

```

set.seed(320)

datos_modelo_7 <- dataset_bancos
train_index_7 <- sample(1:nrow(datos_modelo_2), 0.7 * nrow(datos_modelo_2))
#

# Crear subconjuntos
train_data_7 <- datos_modelo_7[train_index_7, ]
test_data_7 <- datos_modelo_7[-train_index_7, ]

```

```

# Fórmula automática: BAN2 ~ todas las demás variables
form <- as.formula(
  paste("BAN2 ~", paste(setdiff(names(datos_scaled), "BAN2"), collapse = "
+ "))
)

modelo_7 <- randomForest(
  formula = form,
  data = train_data_7,
  ntree = 500, # número de
árboles
  mtry = floor(sqrt(ncol(train_data_7) - 1)), # raíz de
predictores
  importance = TRUE # para ver
importancia de variables
)

print(modelo_7)

x11()
varImpPlot(modelo_7) # gráfico de importancia de variables

predicciones_7 <- predict(
  modelo_7,
  newdata = test_data_7[, setdiff(names(test_data_7),
"BAN2")]
)

y_real7 <- test_data_7$BAN2

mae_7 <- mean(abs(y_real7 - predicciones_7))
rmse_7 <- sqrt(mean((y_real7 - predicciones_7)^2))
r2_7 <- 1 - sum((y_real7 - predicciones_7)^2) / sum((y_real7 -
mean(y_real7))^2)

cat("RMSE:", rmse_7, "\n")
cat("MAE :", mae_7, "\n")
cat("R² :", r2_7, "\n")

```

Análisis predictivo xgboost - MODELO 8

```

set.seed(321)

n <- nrow(datos_scaled)
train_index_8 <- sample(1:n, size = 0.7 * n)

train_data_8 <- datos_scaled[train_index_8, ]
test_data_8 <- datos_scaled[-train_index_8, ]

# Variables predictoras (todas menos BAN2)
predictoras <- setdiff(names(datos_scaled), "BAN2")

# Matriz de entrenamiento
X_train <- as.matrix(train_data_8[, predictoras])
y_train <- train_data_8$BAN2

```

```

# Matriz de test
X_test <- as.matrix(test_data_8[, predictoras])
y_test <- test_data_8$BAN2

# DMatrix (formato especial de xgboost)
dtrain <- xgb.DMatrix(data = X_train, label = y_train)
dtest <- xgb.DMatrix(data = X_test, label = y_test)

params <- list(
  objective = "reg:squarederror", # regresión
  eta = 0.1, # learning rate
  max_depth = 6, # profundidad de árboles
  subsample = 0.8, # muestreo aleatorio
  colsample_bytree = 0.8 # muestreo de columnas
)

modelo_8 <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = 300, # número de árboles
  watchlist = list(train = dtrain, test = dtest),
  print_every_n = 50
)

x11()
vip <- xgb.importance(model = modelo_8)
xgb.plot.importance(vip)

predicciones_8 <- predict(modelo_8, newdata = dtest)

mae_8 <- mean(abs(y_test - predicciones_8))
rmse_8 <- sqrt(mean((y_test - predicciones_8)^2))
r2_8 <- 1 - sum((y_test - predicciones_8)^2) / sum((y_test -
mean(y_test))^2)

cat("RMSE:", rmse_8, "\n")
cat("MAE :", mae_8, "\n")
cat("R² :", r2_8, "\n")

```

Análisis predictivo xgboost no escalado - MODELO 8B

```

set.seed(321)

n <- nrow(dataset_bancos)
train_index_9 <- sample(1:n, size = 0.7 * n)

train_data_9 <- datos_scaled[train_index_9, ]
test_data_9 <- datos_scaled[-train_index_9, ]

# Variables predictoras (todas menos BAN2)
predictoras9 <- setdiff(names(datos_scaled), "BAN2")

# Matriz de entrenamiento
X_train9 <- as.matrix(train_data_9[, predictoras9])

```

```

y_train9 <- train_data_9$BAN2

# Matriz de test
X_test9 <- as.matrix(test_data_9[, predictoras9])
y_test9 <- test_data_9$BAN2

# DMatrix (formato especial de xgboost)
dtrain9 <- xgb.DMatrix(data = X_train9, label = y_train9)
dtest9 <- xgb.DMatrix(data = X_test9, label = y_test9)

params9 <- list(
  objective = "reg:squarederror", # regresión
  eta = 0.1, # learning rate
  max_depth = 6, # profundidad de árboles
  subsample = 0.8, # muestreo aleatorio
  colsample_bytree = 0.8 # muestreo de columnas
)

modelo_9 <- xgb.train(
  params = params9,
  data = dtrain9,
  nrounds = 300, # número de árboles
  watchlist = list(train = dtrain9, test = dtest9),
  print_every_n = 50
)

x11()
vip9 <- xgb.importance(model = modelo_9)
xgb.plot.importance(vip9)

predicciones_9 <- predict(modelo_9, newdata = dtest9)

mae_9 <- mean(abs(y_test9 - predicciones_9))
rmse_9 <- sqrt(mean((y_test9 - predicciones_9)^2))
r2_9 <- 1 - sum((y_test9 - predicciones_9)^2) / sum((y_test9 -
mean(y_test9))^2)

cat("RMSE:", rmse_9, "\n")
cat("MAE :", mae_9, "\n")
cat("R² :", r2_9, "\n")

```

RESUMEN DE RESULTADOS

```

resultados <- data.frame(
  Modelo = c("Modelo Regresión Lineal",
    "Modelo Regresión Lineal + PCE",
    "Modelo GLM",
    "Modelo Redes Neuronales Parametría 1",
    "Modelo Redes Neuronales Parametría 2",
    "Modelo Redes Neuronales Parametría 3",
    "Modelo Random Forest",
    "Modelo XGBOOST"),
  RMSE = c(rmse, rmse_2, rmse_3, rmse_4, rmse_5, rmse_6, rmse_7, rmse_8),
  MAE = c(mae, mae_2, mae_3, mae_4, mae_5, mae_6, mae_7, mae_8),

```

```

R2      = c(r2, r2_2, r2_3, r2_4, r2_5, r2_6, r2_7, r2_8)
)

print(resultados)

# --> CONCLUSIÓN: EL MODELO QUE ARROJA LOS MEJORES RESULTADOS EL REALIZADO
CON XGBOOST

```

Predicción de bancos nuevos para dataset de radios censales sin bancos

```

# Se guarda la columna de id de radios censales
cro_id <- dataset_bancos$cro
cro_id_sin <- dataset_sin_bancos$cro

# Se escala el dataset de cro sin bancos
dataset_sin_bancos_filtrado <- dataset_sin_bancos %>% select(-cro)
datos_sin_scaled <- predict(preproc, dataset_sin_bancos_filtrado)

# Se hacen predicciones con el modelo
datos_para_pred <- datos_sin_scaled %>% select(-BAN2)          #se saca BAN2
pred_scaled <- compute(modelo_4, datos_para_pred)$net.result #los
resultados están entre 0 y 1 por estar escalado

# Se desescalan los resultados
min_BAN2 <- min(dataset_bancos_filtrado$BAN2) # valores mínimos para el
preproceso
max_BAN2 <- max(dataset_bancos_filtrado$BAN2) # valores máximos para el
preproceso

BAN_NVO <- pred_scaled * (max_BAN2 - min_BAN2) + min_BAN2
BAN_NVO <- round(BAN_NVO)                                     # convertir a enteros

BAN_NVO[BAN_NVO < 0] <- 0                                     # valores menores a cero se
vuelven cero

resultados_prediccion <- data.frame(
  cro = cro_id_sin,
  BAN_NVO = BAN_NVO
)

resultados_prediccion

write.csv(resultados_prediccion, "Predicciones.csv", row.names = FALSE)
getwd()

```